



Evasion Defenses

A. M. Sadeghzadeh, Ph.D.

Sharif University of Technology
Computer Engineering Department (CE)
Trustworthy and Secure AI Lab



November 16, 2025

Today's Agenda

1 TRADES

TRADES

TRADES

Theoretically Principled Trade-off between Robustness and Accuracy

Hongyang Zhang*

CMU & TTIC

hongyanz@cs.cmu.edu

Eric P. Xing

CMU & Petuum Inc.

epxing@cs.cmu.edu

Yaodong Yu†

University of Virginia

yy8ms@virginia.edu

Laurent El Ghaoui

UC Berkeley

elghaoui@berkeley.edu

Jiantao Jiao

UC Berkeley

jiantao@eecs.berkeley.edu

Michael I. Jordan

UC Berkeley

jordan@cs.berkeley.edu

Trade Off Analysis

- If adversarial attack is unavoidable, what can we do?
 - We want to show that there is a natural trade-off between accuracy and robustness
 - You can be absolutely robust but useless, or absolutely accurate but very vulnerable
- Intuitively, the existence of trade-off makes sense:
 - You can be very robust, e.g., always claims class 1 regardless what you see. Then you are ultimately robust but not accurate.
 - You can be very accurate, e.g., a perceptron algorithm for linearly separable problems. But you have terrible robustness.
- Our discussion is based on this paper
 - Zhang et al. Theoretically principled trade-off between robustness and accuracy, arXiv: 1901.08573
 - Published in ICML 2019
- There is another very interesting paper
 - Tsipras et al., Robustness May Be at Odds with Accuracy, arXiv: 1805.12152
 - Some observations are quite intriguing.

Main Messages of Zhang et al. 2019

We will focus on Zhang et al. Theoretically principled trade-off between robustness and accuracy, arXiv: 1901.08573.

There are three messages:

- (1) There is an intrinsic trade off between robustness and accuracy
- (2) It is possible to upper bound both terms using a technique called classification-calibrated loss
- (3) You can develop a heuristic algorithm to minimize the empirical risk

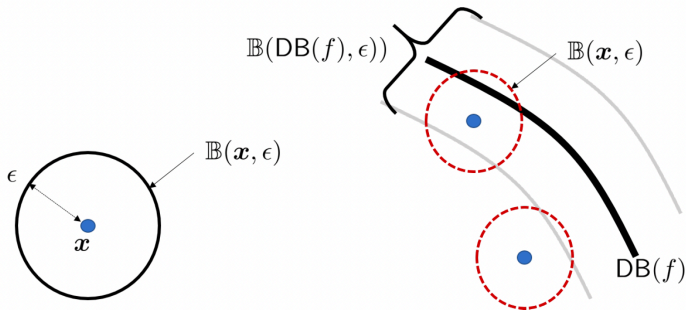
In addition, the paper showed a few very interesting results

- The trade-off optimization generalizes adversarial training
- They outperform defense methods in NIPS 2018 challenges

Notation

- $\mathbf{x} \in \mathcal{X}$: Input data. Random variable $\mathbf{X}$. Realization $\mathbf{x}$.
- $y \in \mathcal{Y} = \{+1, -1\}$: Label. Random variable Y . Realization y .
- Classifier: $f : \mathcal{X} \rightarrow \mathcal{Y}$
- $\mathbb{B}(\mathbf{x}, \epsilon)$ = an ϵ -ball surrounding the point $\mathbf{x}$
 - $\mathbb{B}(\mathbf{x}, \epsilon) = \{\mathbf{x}' \in \mathcal{X} : \|\mathbf{x}' - \mathbf{x}\| \leq \epsilon\}$
- **Decision boundary** of the classifier $\text{DB}(f) = \{\mathbf{x} \in \mathcal{X} : f(\mathbf{x}) = 0\}$.
- **Neighborhood of the decision boundary** $\mathbb{B}(\text{DB}(f), \epsilon)$.
 - $\mathbb{B}(\text{DB}(f), \epsilon) = \{\mathbf{x} \in \mathcal{X} : \exists \mathbf{x}' \in \mathbb{B}(\mathbf{x}, \epsilon) \text{ s.t. } f(\mathbf{x})f(\mathbf{x}') \leq 0\}$
 - Basically: The band surrounding the decision boundary
 - Pick a point $\mathbf{x}$. If $\mathbf{x}$ is inside the band, then you can find $\mathbf{x}'$ with the epsilon ball of $\mathbf{x}$, where $f(\mathbf{x}) = +1$ and $f(\mathbf{x}') = -1$.
 - If $\mathbf{x}$ is outside the band, then within the same epsilon ball you will not be able to find a point that is predicted with an opposite label.

Notation



Accuracy and Robustness

Natural Classification Error

$$\mathcal{R}_{\text{nat}}(f) = \mathbb{E}_{(\mathbf{X}, Y) \sim \mathcal{D}} \mathbb{I}\{f(\mathbf{X})Y \leq 0\}. \quad (6)$$

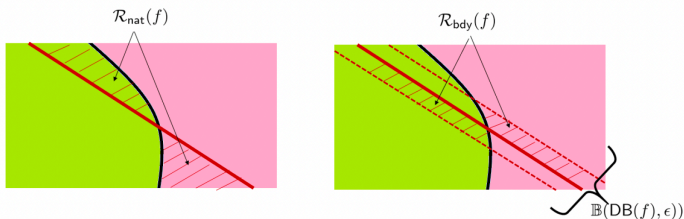
- You pick an input $\mathbf{X}$.
- The prediction is $f(\mathbf{X})$.
- You compare with the true label Y .
- If mismatch, then $f(\mathbf{X})Y \leq 0$.
- The indicator function $\mathbb{I}$ will tell you whether this is indeed a mistake.
- Then you average over all the possible inputs $\mathbf{X} \sim D$.
- This will tell you the amount of error made by your classifier.
- Of course, you want this natural error as small as possible.
- $1 - \mathcal{R}_{\text{nat}}(f)$ is the **natural accuracy**. You want it as high as possible.

Accuracy and Robustness

Boundary Classification Error

$$\mathcal{R}_{\text{bdy}}(f) = \mathbb{E}_{(\mathbf{X}, Y) \sim \mathcal{D}} \mathbb{I}\{\mathbf{X} \in \mathbb{B}(\text{DB}(f), \epsilon), f(\mathbf{X})Y > 0\} \quad (7)$$

- $\mathbf{X} \in \mathbb{B}(\text{DB}(f))$ means the point $\mathbf{X}$ is inside the band.
- $f(\mathbf{X})Y > 0$ means that $\mathbf{X}$ is correctly classified.
- So, $\mathcal{R}_{\text{bdy}}(f)$ is anything inside the band **and** is correctly classified.

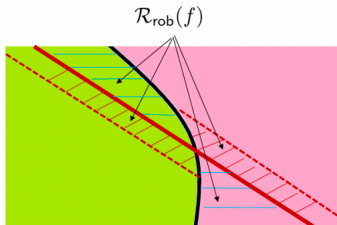


Accuracy and Robustness

Robust Classification Error

$$\mathcal{R}_{\text{rob}}(f) = \mathcal{R}_{\text{nat}}(f) + \mathcal{R}_{\text{bdy}}(f) \quad (8)$$

- This is the sum of the two error: Anything that you have already made mistake (natural error), plus anything that you will likely to make mistake (boundary error)



Robust vs. Non-Robust Decision Boundaries

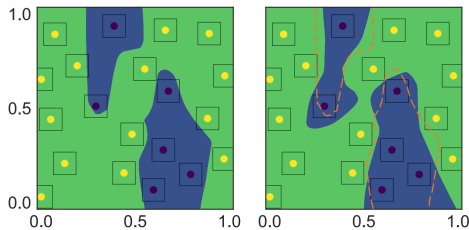
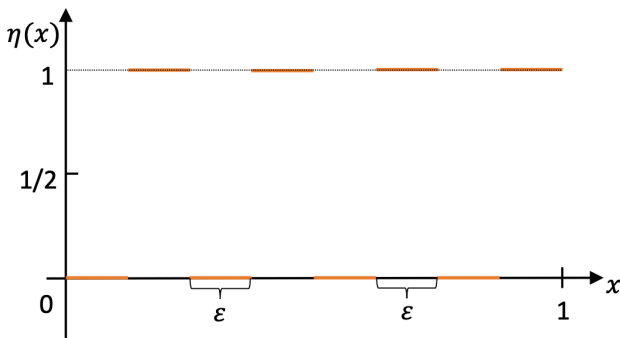


Figure 1: **Left figure:** decision boundary learned by natural training method. **Right figure:** decision boundary learned by our adversarial training method, where the orange dotted line represents the decision boundary in the left figure. It shows that both methods achieve zero natural training error, while our adversarial training method achieves better robust training error than the natural training method.

Example

Example. Consider the case $(X, Y) \sim \mathcal{D}$, where the marginal distribution over the instance space is a uniform distribution over $[0, 1]$, and for $k = 0, 1, \dots, \lceil \frac{1}{2\epsilon} \rceil - 1$,

$$\eta(x) := \Pr(Y = 1 \mid X = x) = \begin{cases} 0, & x \in [2k\epsilon, (2k+1)\epsilon), \\ 1, & x \in ((2k+1)\epsilon, (2k+2)\epsilon]. \end{cases} \quad (2)$$



Example

	Bayes Optimal Classifier	All-One Classifier
$\mathcal{R}_{\text{nat}}$	0 (optimal)	1/2
$\mathcal{R}_{\text{bdy}}$	1	0
$\mathcal{R}_{\text{rob}}$	1	1/2 (optimal)

- Because you know the posterior distribution, Bayesian optimal classifier (based on MAP) will be exactly the same as $\eta(x)$. So $\mathcal{R}_{\text{nat}} = 0$ and it is optimal.
- The boundary error is 1, because the band is just the entire internal
- You can choose an all-one classifier. You always claim 1.
- This is a bad classifier in terms of natural accuracy. Half is correct, half is wrong.
- But the robustness error is actually better than Bayesian optimal.

Upper Bounding the Error

- After defining how to measure robustness, we can now ask about the **fundamental limit** of $\mathcal{R}_{\text{rob}}(f)$.
- The approach proposed by the paper is to
 - Define $\mathcal{R}_{\text{nat}}^* = \min_f \mathcal{R}_{\text{nat}}(f)$ be the best classifier (based on minimize the natural error).
 - We want to upper bound $\mathcal{R}_{\text{rob}}(f) - \mathcal{R}_{\text{nat}}^*$, so that we know $\mathcal{R}_{\text{rob}}(f)$ is more than $\mathcal{R}_{\text{nat}}^*$ by some maximum amount.
 - If we can find such upper bound, then we can perhaps minimizing the upper bound.
- Let us first state the theorem, and discuss the equations.
- We will skip the details. You should read the paper.

Theorem (Zhang et al. 2019 Theorem 3.1)

$$\mathcal{R}_{\text{rob}}(f) - \mathcal{R}_{\text{nat}}^* \leq \psi^{-1}(\mathcal{R}_{\phi}(f) - \mathcal{R}_{\phi}^*) + \mathbb{E} \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')f(\mathbf{X})/\lambda). \quad (9)$$

Basic Argument

The theorem states that

$$\mathcal{R}_{\text{rob}}(f) - \mathcal{R}_{\text{nat}}^* \leq \psi^{-1}(\mathcal{R}_{\phi}(f) - \mathcal{R}_{\phi}^*) + \mathbb{E} \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')f(\mathbf{X})/\lambda). \quad (10)$$

- The basic argue goes as follows.

$$\mathcal{R}_{\text{rob}}(f) - \mathcal{R}_{\text{nat}}^*$$

$$\stackrel{(a)}{=} \mathcal{R}_{\text{nat}}(f) - \mathcal{R}_{\text{nat}}^* + \mathcal{R}_{\text{bdy}}(f) \quad \text{because } \mathcal{R}_{\text{rob}} = \mathcal{R}_{\text{nat}} + \mathcal{R}_{\text{bdy}}$$

$$\stackrel{(b)}{\leq} \psi^{-1}(\mathcal{R}_{\phi}(f) - \mathcal{R}_{\phi}^*) + \mathcal{R}_{\text{bdy}}(f) \quad \text{using surrogate loss } \psi$$

$$\stackrel{(c)}{=} \psi^{-1}(\mathcal{R}_{\phi}(f) - \mathcal{R}_{\phi}^*) + \mathbb{P}[\mathbf{X} \in \mathbb{B}(\text{DB}(f), \epsilon), f(\mathbf{X})Y > 0]$$

$$\stackrel{(d)}{\leq} \psi^{-1}(\mathcal{R}_{\phi}(f) - \mathcal{R}_{\phi}^*) + \mathbb{E} \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')f(\mathbf{X})/\lambda), \quad \text{for some } \lambda > 0.$$

Let us talk about these steps one by one.

Step (b)

$$\begin{aligned}\mathcal{R}_{\text{rob}}(f) - \mathcal{R}_{\text{nat}}^* &= \mathcal{R}_{\text{nat}}(f) - \mathcal{R}_{\text{nat}}^* + \mathcal{R}_{\text{bdy}}(f) \\ &\stackrel{(b)}{\leq} \psi^{-1}(\mathcal{R}_{\phi}(f) - \mathcal{R}_{\phi}^*) + \mathcal{R}_{\text{bdy}}(f)\end{aligned}$$

- In principle, $\mathcal{R}_{\text{nat}}(f)$ should be measured using $\mathcal{R}_{\text{nat}}(f) = \mathbb{E}_{(\mathbf{X}, Y) \sim \mathcal{D}} \mathbb{I}\{f(\mathbf{X})Y \leq 0\}$.
- The 0-1 loss is not differentiable, and poses difficulty in analysis.
- One way to handle that is to replace the 0-1 loss by the so-called classification-calibrated surrogate loss ².
- Surrogate loss comes with a pair of functions ϕ and ψ .
- Here are some examples

Loss	$\phi(\alpha)$	$\psi(\theta)$
Hinge	$\max\{1 - \alpha, 0\}$	θ
Sigmoid	$1 - \tanh(\alpha)$	θ
Exponential	$\exp(-\alpha)$	$1 - \sqrt{1 - \theta^2}$
Logistic	$\log_2(1 + \exp(-\alpha))$	$\psi_{\log}(\theta)$

²See Peter L Bartlett, Michael I Jordan, and Jon D McAuliffe. Convexity, classification, and risk bounds. *Journal of the American Statistical Association*, 24 / 29

Step (b)

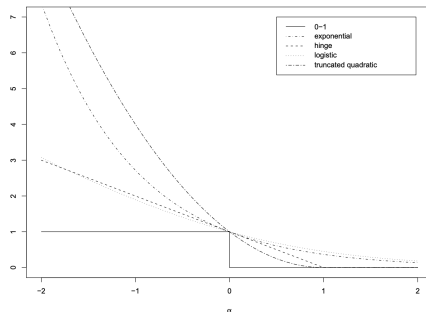


Figure 1: A plot of the 0-1 loss function and surrogates corresponding to various practical classifiers. These functions are plotted as a function of the margin $\alpha = yf(x)$. Note that a classification error is made if and only if the margin is negative; thus the 0-1 loss is a step function that is equal to one for negative values of the abscissa. The curve labeled “logistic” is the negative log likelihood, or deviance, under a logistic regression model; “hinge” is the piecewise-linear loss used in the support vector machine; and “exponential” is the exponential loss used by the Adaboost algorithm. The deviance is scaled so as to majorize the 0-1 loss; see Lemma 9.

Peter L Bartlett, Michael I Jordan, and Jon D McAuliffe. Convexity, classification, and risk bounds. *Journal of the American Statistical Association*, 101(473):138156, 2006.

Step (c) and (d)

Steps (c) and (d):

- (c) is just the definition of the $\mathcal{R}_{\text{bdy}}(f)$
- (d) follows from this

$$\begin{aligned} & \mathbb{P}[\mathbf{X} \in \mathbb{B}(\text{DB}(f), \epsilon), f(\mathbf{X})Y > 0] \\ & \leq \mathbb{P}[\mathbf{X} \in \mathbb{B}(\text{DB}(f), \epsilon)] \quad \text{former is a subset of latter} \\ & = \mathbb{E} \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \mathbb{I}\{f(\mathbf{X}') \neq f(\mathbf{X})\} \\ & = \mathbb{E} \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \mathbb{I}\{f(\mathbf{X}')f(\mathbf{X})/\lambda < 0\} \quad \text{for all } \lambda \\ & \leq \mathbb{E} \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')f(\mathbf{X})/\lambda) \end{aligned}$$

- You can think of λ as a regularization parameter
- Theorem 3.1 holds for all λ
- Theorem 3.2 says that in order for theorem to hold, you need to carefully pick a λ

Optimization

- The theorem above suggest an optimization to minimize $\mathcal{R}_{\text{rob}}(f) - \mathcal{R}_{\text{nat}}^*$:

$$\min_f \underbrace{\psi^{-1}(\mathcal{R}_{\phi}(f) - \mathcal{R}_{\phi}^*)}_{\text{accuracy}} + \underbrace{\mathbb{E} \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')f(\mathbf{X})/\lambda)}_{\text{robustness}}$$

- You can replace the first term by the empirical risk $\phi(f(\mathbf{X})Y)$
- This will give you

$$\min_f \mathbb{E} \left\{ \underbrace{\phi(f(\mathbf{X})Y)}_{\text{accuracy}} + \underbrace{\max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')f(\mathbf{X})/\lambda)}_{\text{robustness}} \right\}$$

- There is a regularization parameter λ

What do you gain?

Let us look at this optimization again:

$$\min_f \mathbb{E} \left\{ \underbrace{\phi(f(\mathbf{X})Y)}_{\text{accuracy}} + \underbrace{\max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')f(\mathbf{X})/\lambda)}_{\text{robustness}} \right\}$$

- This optimization is a trade-off between **accuracy** and **robustness**
- Recall **adversarial training** (Madry et al.)

$$\min_f \mathbb{E} \left\{ \max_{\mathbf{X}' \in \mathbb{B}(\mathbf{X}, \epsilon)} \phi(f(\mathbf{X}')Y) \right\}$$

- It is an upper bound of $\mathcal{R}_{\text{rob}}(f)$
- The upper bound offered by the trade-off formulation is tighter

TRADES Algorithm

Algorithm 1 Adversarial training by TRADES

```

1: Inputs: Step sizes  $\eta_1$  and  $\eta_2$ , batch size  $m$ , number of iterations  $K$  in inner optimization, network architecture parametrized by  $\theta$ 
2: Randomly initialize network  $f_\theta$ , or initialize network with pre-trained configuration
3: repeat
4:   Read mini-batch  $B = \{\mathbf{x}_1, \dots, \mathbf{x}_m\}$  from training set
5:   for  $i = 1, \dots, m$  (in parallel) do
6:      $\mathbf{x}'_i \leftarrow \mathbf{x}_i + 0.001 \cdot \mathcal{N}(0, I)$ 
7:     for  $k = 1, \dots, K$  do
8:        $\mathbf{x}'_i \leftarrow \Pi_{\mathbb{B}(\mathbf{x}_i, \epsilon)}(\eta_1 \cdot \text{sign}(\nabla_{\mathbf{x}'_i} \mathcal{L}(f_\theta(\mathbf{x}_i), f_\theta(\mathbf{x}'_i))) + \mathbf{x}'_i)$ 
9:     end for
10:  end for
11:   $\theta \leftarrow \theta - \eta_2 \frac{1}{m} \sum_{i=1}^m \nabla_\theta [\mathcal{L}(f_\theta(\mathbf{x}_i), y_i) + \frac{1}{\lambda} \mathcal{L}(f_\theta(\mathbf{x}_i), f_\theta(\mathbf{x}'_i))]$ 
12: until training converged

```

Results on CIFAR10 and MNIST

Table 5: Comparisons of TRADES with prior defense models under white-box attacks.

Defense	Defense type	Under which attack	Dataset	Distance	$\mathcal{A}_{\text{nat}}(f)$	$\mathcal{A}_{\text{rob}}(f)$
[BRRG18]	gradient mask	[ACW18]	CIFAR10	0.031 (ℓ_{∞})	-	0%
[MLW+18]	gradient mask	[ACW18]	CIFAR10	0.031 (ℓ_{∞})	-	5%
[DAL+18]	gradient mask	[ACW18]	CIFAR10	0.031 (ℓ_{∞})	-	0%
[SKN+18]	gradient mask	[ACW18]	CIFAR10	0.031 (ℓ_{∞})	-	9%
[NKM17]	gradient mask	[ACW18]	CIFAR10	0.015 (ℓ_{∞})	-	15%
[WSMK18]	robust opt.	FGSM ²⁰ (PGD)	CIFAR10	0.031 (ℓ_{∞})	27.07%	23.54%
[MMS+18]	robust opt.	FGSM ²⁰ (PGD)	CIFAR10	0.031 (ℓ_{∞})	87.30%	47.04%
[ZSLG16]	regularization	FGSM ²⁰ (PGD)	CIFAR10	0.031 (ℓ_{∞})	94.64%	0.15%
[KGB17]	regularization	FGSM ²⁰ (PGD)	CIFAR10	0.031 (ℓ_{∞})	85.25%	45.89%
[RDV17]	regularization	FGSM ²⁰ (PGD)	CIFAR10	0.031 (ℓ_{∞})	95.34%	0%
TRADES ($1/\lambda = 1$)	regularization	FGSM ^{1,000} (PGD)	CIFAR10	0.031 (ℓ_{∞})	88.64%	48.90%
TRADES ($1/\lambda = 6$)	regularization	FGSM ^{1,000} (PGD)	CIFAR10	0.031 (ℓ_{∞})	84.92%	56.43%
TRADES ($1/\lambda = 1$)	regularization	FGSM ²⁰ (PGD)	CIFAR10	0.031 (ℓ_{∞})	88.64%	49.14%
TRADES ($1/\lambda = 6$)	regularization	FGSM ²⁰ (PGD)	CIFAR10	0.031 (ℓ_{∞})	84.92%	56.61%
TRADES ($1/\lambda = 1$)	regularization	DeepFool (ℓ_{∞})	CIFAR10	0.031 (ℓ_{∞})	88.64%	59.10%
TRADES ($1/\lambda = 6$)	regularization	DeepFool (ℓ_{∞})	CIFAR10	0.031 (ℓ_{∞})	84.92%	61.38%
TRADES ($1/\lambda = 1$)	regularization	LBFGSAttack	CIFAR10	0.031 (ℓ_{∞})	88.64%	84.41%
TRADES ($1/\lambda = 6$)	regularization	LBFGSAttack	CIFAR10	0.031 (ℓ_{∞})	84.92%	81.58%
TRADES ($1/\lambda = 1$)	regularization	MI-FGSM	CIFAR10	0.031 (ℓ_{∞})	88.64%	51.26%
TRADES ($1/\lambda = 6$)	regularization	MI-FGSM	CIFAR10	0.031 (ℓ_{∞})	84.92%	57.95%
TRADES ($1/\lambda = 1$)	regularization	C&W	CIFAR10	0.031 (ℓ_{∞})	88.64%	84.03%
TRADES ($1/\lambda = 6$)	regularization	C&W	CIFAR10	0.031 (ℓ_{∞})	84.92%	81.24%
[SKC18]	gradient mask	[ACW18]	MNIST	0.005 (ℓ_2)	-	55%
[MMS+18]	robust opt.	FGSM ⁴⁰ (PGD)	MNIST	0.3 (ℓ_{∞})	99.36%	96.01%
TRADES ($1/\lambda = 6$)	regularization	FGSM ^{1,000} (PGD)	MNIST	0.3 (ℓ_{∞})	99.48%	95.60%
TRADES ($1/\lambda = 6$)	regularization	FGSM ⁴⁰ (PGD)	MNIST	0.3 (ℓ_{∞})	99.48%	96.07%
TRADES ($1/\lambda = 6$)	regularization	C&W	MNIST	0.005 (ℓ_2)	99.48%	99.46%

References and Resources I

- Adversarial Training (AT) was first introduced by Goodfellow et al. [5], however the adversary utilized was quite weak. Madry et al. [7] were the first to use the PGD attack to improve their results. As a consequence of their wildly successful paper, their method of Adversarial Training is now considered to be the standard version.
- Athalye, Carlini, and Wagner [1] identified obfuscated gradients and broke 7 out of 9 defenses [2, 6, 4, 9, 8] from ICLR 2018. Only Adversarial Training [7] managed to retain its performance and as a result, emerged as the most widely used defense mechanism against adversarial examples.
- Zhang et al. [10] identify a trade-off between robustness and accuracy that serves as a guiding principle in the design of defenses against adversarial examples.
- Currently, different variants of Adversarial Training dominate Robust Bench [3].

References and Resources II

- [1] Anish Athalye, Nicholas Carlini, and David Wagner. **Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples**, 2018. URL <https://arxiv.org/abs/1802.00420>.
- [2] Jacob Buckman, Aurko Roy, Colin Raffel, and Ian Goodfellow. Thermometer Encoding: One Hot Way to Resist Adversarial Examples. In **International Conference on Learning Representations**, 2018. URL <https://openreview.net/forum?id=S18Su--CW>.
- [3] Francesco Croce, Maksym Andriushchenko, Vikash Sehwal, Edoardo Debenedetti, Nicolas Flammarion, Mung Chiang, Prateek Mittal, and Matthias Hein. RobustBench: A Standardized Adversarial Robustness Benchmark, 2021. URL <https://arxiv.org/abs/2010.09670>.
- [4] Guneet S. Dhillon, Kamyar Azizzadenesheli, Zachary C. Lipton, Jeremy Bernstein, Jean Kossaifi, Aran Khanna, and Anima Anandkumar. Stochastic activation pruning for robust adversarial defense, 2018. URL <https://arxiv.org/abs/1803.01442>.
- [5] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and Harnessing Adversarial Examples, 2015. URL <https://arxiv.org/abs/1412.6572>.

References and Resources III

- [6] Chuan Guo, Mayank Rana, Moustapha Cisse, and Laurens van der Maaten. Countering Adversarial Images using Input Transformations, 2018. URL <https://arxiv.org/abs/1711.00117>.
- [7] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. **Towards Deep Learning Models Resistant to Adversarial Attacks**, 2019. URL <https://arxiv.org/abs/1706.06083>.
- [8] Yang Song, Taesup Kim, Sebastian Nowozin, Stefano Ermon, and Nate Kushman. PixelDefend: Leveraging Generative Models to Understand and Defend against Adversarial Examples, 2018. URL <https://arxiv.org/abs/1710.10766>.
- [9] Cihang Xie, Jianyu Wang, Zhishuai Zhang, Zhou Ren, and Alan Yuille. Mitigating Adversarial Effects Through Randomization, 2018. URL <https://arxiv.org/abs/1711.01991>.
- [10] Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric P. Xing, Laurent El Ghaoui, and Michael I. Jordan. **Theoretically Principled Trade-off between Robustness and Accuracy**, 2019. URL <https://arxiv.org/abs/1901.08573>.